

PSE Traversal Agent

Implementierungsspezifikation fuer einen post-symbolischen
Traversierungsagenten mit ueberpruefbarer Historie

Sebastian Klemm
Independent Researcher, Germany

Version 0.1 – Agentenimplementierung, Mai 2026

Zusammenfassung

Dieses Dokument transformiert das Topologische Traversierungsframework v0.3 in eine implementierbare Spezifikation fuer einen Coding-Agenten. Ziel ist kein weiteres Theoriepapier, sondern ein belastbarer Bauplan fuer eine neue Schicht ueber PSE: einen post-symbolischen Traversierungsagenten mit ueberpruefbarer Historie. Der Agent modelliert Aufgaben als Feld-Cubes, konstruiert Freiheitsgradgraphen, plant Collapse-Schritte, schneidet ungueltige Pfade ab, fuehrt Solver kontrolliert aus, bindet Kandidaten an Evidence und committet nur nachvollziehbare Strukturereignisse ueber den bestehenden PSE-Kern.

Die Spezifikation ist bewusst inkrementell: Phase 1 implementiert nur Typen, deterministische Serialisierung, ProblemSpec-Parsing, DoFGraph, FieldCube, CollapsePlan und PathExcision. Phase 2 koppelt Kandidaten an PSE-Observations und SemanticCrystals. Phase 3 fuegt Norm-Lifecycle, Dual-Fabric-Gating und Collapse Certificates hinzu. Phase 4 ergaenzt einen Agent Harness fuer LLMs, Tools oder menschliche Solver. Alle Phasen muessen replaybar, testbar und fail-closed sein.

Inhaltsverzeichnis

1	Zweck und Erfolgskriterium	2
2	Ausgangslage im Repository	2
3	Architekturentscheidung	3
3.1	Ein Repository, getrennte Crates	3
3.2	Minimaler Start	3
4	Systemrollen	4
5	Nicht-Ziele	4
6	Kernmodell	4
6.1	ProblemSpec	4
6.2	Dimensionen und Wertedomänen	5
6.3	FieldCube	5
6.4	DoFGraph	6
7	Operatoren	7
7.1	Operator Registry	7
7.2	Pflichtoperatoren fuer MVP	7

8 Hypercube- und Collapse-Planung	8
8.1 Hypercube-Konstruktion	8
8.2 CollapsePlan	8
8.3 Ordering Policy	9
9 Optionalitaet und Pfadexzision	9
9.1 Formale vs. operative Option	9
9.2 Algorithmus	10
10 Carrier und Phase-Ladder	10
10.1 Migration	10
11 Gate, Dual-Fabric und Fail-Closed	11
11.1 GateReport	11
11.2 Fail-Closed-Regel	11
11.3 Dual-Fabric im MVP	11
12 PSE-Bruecke	12
12.1 Prinzip	12
12.2 PseMacroStepCommitter	12
13 Evidence, Replay und Canonical Output	13
13.1 Canonical Serialization	13
13.2 Run Descriptor	13
14 CLI fuer den Coding-Agenten	13
15 Beispiel-ProblemSpec	14
16 Phasenplan fuer die Implementierung	14
16.1 Phase 0: Repository-Audit	14
16.2 Phase 1: pse-traverse MVP	15
16.3 Phase 2: CLI	15
16.4 Phase 3: PSE-Bruecke	15
16.5 Phase 4: Normen, Dual-Fabric, Collapse Certificate	15
16.6 Phase 5: Agent Harness	15
17 Testplan	16
18 Definition of Done	16
18.1 MVP-konform	16
18.2 v0.1-konform	16
19 Masterprompt fuer den Coding-Agenten	17
A Maschinenlesbarer Minimalvertrag	17
B Glossar	18
C Schlussbemerkung	18

1 Zweck und Erfolgskriterium

Der Zweck dieser Spezifikation ist die Umsetzung eines konkreten Systems im Repository. Das System soll nicht bloss neue Begriffe einfuehren, sondern den bestehenden PSE-Maschinenraum um eine Traversierungs- und Agentenschicht erweitern.

Kernsatz

Ein PSE Traversal Agent ist eine deterministische Steuerungsschicht, die strukturierte Problemraeume in topologische Freiheitsgradraeume ueberfuehrt, diese Raeume schrittweise traversiert, Kandidaten gated, Fehlschlaege in Verfeinerungen umwandelt und jeden relevanten Strukturfortschritt als replaybares Evidence-Objekt oder PSE-Crystal absichert.

Der Erfolg ist nicht erreicht, wenn ein weiteres PDF oder eine weitere abstrakte Architektur existiert. Erfolg ist erreicht, wenn ein Coding-Agent im Repository folgende Minimalfaehigkeiten implementieren kann:

1. eine ProblemSpec aus Datei laden und canonical serialisieren;
2. daraus einen FieldCube mit Dimensionen, Werten, Constraints, Pfaden und Carriern konstruieren;
3. einen DoFGraph und einen CollapsePlan erzeugen;
4. formale Optionen von operativen Optionen unterscheiden;
5. Pfadexzisionen als GapReport oder spaeter GapCrystal ausgeben;
6. Kandidaten ueber ein Gate pruefen und fail-closed behandeln;
7. PSE als Commit- und Evidence-Kern anbinden, statt Crystal-IDs frei zu erfinden;
8. alle Outputs deterministisch, sortiert und replaybar erzeugen.

2 Ausgangslage im Repository

PSE ist bereits ein geeigneter Unterbau. Das README beschreibt PSE als Streaming Computation Engine, die Beobachtungen in eine 5D-topologische Graphstruktur projiziert, Mandorla-Kohaerenz prueft, durch ein achtfaches Kairos-Gate filtert, duale Cascade-Consensus-Pruefungen ausfuehrt, optional gegen Surrogatstroeme falsifiziert und erfolgreiche Ereignisse als content-addressed Crystals emittiert.

Die bestehende Architektur ist fuer diese Spezifikation ausreichend stark, weil PSE bereits folgende Schichten besitzt:

PSE-Komponente	Rolle fuer den Traversal Agent
pse-types	Gemeinsame Typen wie Observation, SemanticCrystal, GateSnapshot, Config.
pse-graph	Projektion von Observations in einen PersistentGraph und 5D-Embeddings.
pse-core	Engine-Orchestrator, macro_step, GlobalState, AdaptiveCalibrator, Operatoralgebra, Falsifier.
pse-evidence	Evidence Chain, Crystal-Konstruktion, Content Addressing.

pse-replay	Deterministische Replay- und Vergleichslogik.
pse-topology	Laplacian, Fiedler, Betti, Spectral Gap und weitere Topologiemetriken.
pse-memory	Pattern-Memory fuer wiedererkennbare Strukturformen.
pse-navigator	TRITON- / SimplexMesh- / Singularitaetsnavigation als spaetere Suchraumhilfe.
pse-store	Persistenz fuer Crystals und Evidence.

Die neue Schicht soll diese Komponenten nicht duplizieren. Sie soll den Raum vor dem Commit strukturieren.

3 Architekturentscheidung

3.1 Ein Repository, getrennte Crates

Da im neuen Repository eine Kopie von PSE liegt, wird die Weiterentwicklung als Monorepo empfohlen. Die Traversal-Schicht wird nicht in pse-core hineingemischt, sondern als neue Crates neben dem bestehenden Kern angelegt.

Listing 1: Zielstruktur im Repository

```
crates/
  pse-types
  pse-graph
  pse-core
  pse-evidence
  pse-replay
  pse-topology
  pse-memory
  ...
  pse-traverse # neu: Traversierung, FieldCube, CollapsePlan
  pse-dof # optional spaeter: DoFGraph, Couplings, Singularitaeten
  pse-norms # optional spaeter: Normen, Constraint-Lattice, WEAVE
  pse-agent # optional spaeter: Solver-/LLM-/Tool-Harness

tools/
  pse-traverse-cli # neu: CLI fuer ProblemSpec -> Plan -> Report
```

3.2 Minimaler Start

Der erste Implementierungsschritt ist crates/pse-traverse. Diese Crate enthaelt zu- naechst auch DoF-, Norm- und Agent-Vorformen. Erst wenn die Oberflaeche stabil ist, werden diese Bereiche in eigene Crates ausgelagert.

Nicht tun

Der Coding-Agent soll nicht sofort ein volles Agentensystem bauen. Kein Netzwerk, keine LLM-API, keine Tauri-GUI, keine neue Persistenzschicht, kein Ersatz fuer pse-core. Zuerst wird ein deterministischer, getesteter Traversal-Kern gebaut.

4 Systemrollen

Rolle	Definition
PSE	Commit-, Evidence-, Replay-, Gate- und Crystal-Kern. PSE entscheidet, welche Strukturereignisse als Crystals committed werden.
Traversal Agent	Steuerungsschicht ueber PSE. Modelliert Problemraeume, plant Traversierungen, fuehrt Solver aus, erzeugt Kandidaten und Evidence.
Solver	Austauschbarer Loesungsoperator: deterministisch, template-basiert, LLM-basiert, Tool-basiert oder menschlich.
Gate	Pruefschicht vor Commit. Kombiniert lokale Constraints, Dual-Fabric, PSE-Gate, Falsifier und Replay-Anforderungen.
Coagula	Fehlerverarbeitung. Ein Fail wird nicht ignoriert, sondern in Verfeinerung, Pfadexzision, Normluecke, Boundary oder Abbruch uebersetzt.

5 Nicht-Ziele

Die folgenden Punkte sind explizit ausgeschlossen:

1. keine verdeckte Manipulationsmaschine;
2. kein Claim unbeschraenkter Kontextvergroesserung;
3. keine Ersetzung von PSE-Crystals durch frei erfundene IDs;
4. keine nichtdeterministische Ausgabe durch ungeordnete HashMaps;
5. keine ungated Oracle-Emission;
6. keine automatische Wahrheitserklaerung bei 51 Prozent Constraints;
7. keine Vermischung von PSE-Event-5D und Artefakt-/Problem-5D ohne expliziten Projektor.

6 Kernmodell

6.1 ProblemSpec

Der Eingang ist keine freie Prompt-Zeichenkette, sondern eine normierte ProblemSpec. Ein Prompt darf eine ProblemSpec erzeugen, ersetzt sie aber nicht.

Listing 2: ProblemSpec-Kern

```
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct ProblemSpec {
    pub id: String,
    pub title: String,
    pub objective: String,
    pub domain: String,
```

```

pub inputs: Vec<InputRef>,
pub constraints: Vec<ConstraintSpec>,
pub dimensions: Vec<DimensionSpec>,
pub desired_outputs: Vec<OutputSpec>,
pub risk_policy: RiskPolicy,
pub replay: ReplayPolicy,
pub metadata: BTreeMap<String, String>,
}

```

6.2 Dimensionen und Wertedomänen

Eine Dimension ist ein Freiheitsgrad. Sie kann diskret, kontinuierlich, boolesch, hierarchisch oder durch einen externen Enumerator definiert sein.

Listing 3: DimensionSpec

```

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct DimensionSpec {
    pub id: String,
    pub label: String,
    pub kind: DimensionKind,
    pub values: ValueDomain,
    pub required: bool,
    pub source: DimensionSource,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum DimensionKind {
    Boolean,
    Enum,
    Ordinal,
    Continuous,
    Hierarchical,
    Derived,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum ValueDomain {
    Boolean,
    Enum(Vec<String>),
    Range { min: f64, max: f64, step: Option<f64> },
    Tree(Vec<TreeValue>),
    External { resolver: String },
}

```

6.3 FieldCube

Der FieldCube ist die implementierbare Form des erweiterten Hypercube. Er enthaelt nicht nur Dimensionen und Werte, sondern auch Constraints, Kopplungen, Pfade, Carrier, Kohärenzfunktionen und Evidence.

Listing 4: FieldCube

```

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct FieldCube {
    pub id: String,
    pub problem_id: String,
}

```

```

pub dimensions: BTreeMap<String, DimensionSpec>,
pub constraints: BTreeMap<String, ConstraintSpec>,
pub couplings: Vec<Coupling>,
pub paths: Vec<PathSpec>,
pub carriers: Vec<CarrierState>,
pub evidence: Vec<EvidenceRef>,
pub topology: TopologySummary,
pub created_by: String,
}

```

FieldCube-Invariante

Alle Maps, Sets und Listen, die canonical output beeinflussen, muessen vor Serialisierung deterministisch sortiert sein. Kein Output darf von Iterationsreihenfolgen nichtdeterministischer Datenstrukturen abhaengen.

6.4 DoFGraph

Der DoFGraph ist der Arbeitsgraph des Agents. Knoten sind Freiheitsgrade, Constraint-Knoten, Evidence-Knoten, Kandidaten, Boundaries und Merge-Punkte.

Listing 5: DoFGraph

```

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct DoFGraph {
    pub nodes: BTreeMap<NodeId, DoFNode>,
    pub edges: Vec<DoFEdge>,
    pub topology: TopologySummary,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq, Eq, PartialOrd, Ord)]
pub struct NodeId(pub String);

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct DoFNode {
    pub id: NodeId,
    pub kind: DoFNodeKind,
    pub label: String,
    pub weight: f64,
    pub status: NodeStatus,
    pub evidence: Vec<EvidenceRef>,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum DoFNodeKind {
    Dimension,
    Constraint,
    Evidence,
    Candidate,
    Boundary,
    Merge,
    Commit,
}

```

7 Operatoren

7.1 Operator Registry

Jeder Operator muss registriert werden. Das ist ein Kernschutz gegen Begriffsproliferation. Ein Konzept ist erst implementierungsrelevant, wenn es eine Signatur besitzt.

Listing 6: Operator Registry Schema

```
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct OperatorSpec {
    pub id: String,
    pub version: String,
    pub input_type: String,
    pub output_type: String,
    pub determinism_class: DeterminismClass,
    pub admissibility_predicate: String,
    pub evidence_schema: String,
    pub failure_modes: Vec<String>,
    pub replay_requirements: Vec<String>,
    pub status: OperatorStatus,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum DeterminismClass {
    Pure,
    Boundary,
    Oracle,
    Human,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum OperatorStatus {
    Defined,
    Derived,
    ModelSpecific,
    External,
    Open,
}
```

7.2 Pflichtoperatoren fuer MVP

Operator	Signatur	Aufgabe
project	ProblemSpec -> FieldCube	Erzeugt den strukturierten Moeglichkeitsraum.
build_dof_graph	FieldCube -> DoFGraph	Erzeugt den Freiheitsgradgra- phen.
plan_collapse	DoFGraph -> CollapsePlan	Erzeugt deterministische Bear- beitungsreihenfolge.
detect_path_excision	FieldCube -> Vec<PathExcision>	Unterscheidet formale und ope- rative Optionen.
solve	ContextProjection -> CandidateSet	Erzeugt Kandidaten. Im MVP nur Null-/Template-Solver.

gate	Candidate -> GateReport	Prueft Constraints, Evidence, Dual-Fabric optional.
coagula	GateFail -> Refinement	Wandelt Fail in naechste Struk- turaktion um.
commit	Candidate -> CommitReport	Bindet Candidate an PSE oder Evidence-Report.

8 Hypercube- und Collapse-Planung

8.1 Hypercube-Konstruktion

Der Algorithmus muss aus einer ProblemSpec einen FieldCube bauen:

1. Dimensionen aus ProblemSpec.dimensions uebernehmen.
2. Constraints normalisieren und IDs deterministisch ableiten.
3. Kopplungen aus expliziten Constraint-Referenzen ableiten.
4. Default-Pfade zwischen Startzustand und Zieldimensionen erzeugen.
5. Carrier fuer Analyse, Solve, Verify, Commit und Coagula initialisieren.
6. TopologySummary berechnen: Knoten, Kanten, Komponenten, Zyklenindikator, einfache Zentralitaet.

Listing 7: FieldCubeBuilder Trait

```
pub trait FieldCubeBuilder {
    fn build(&self, spec: &ProblemSpec) -> Result<FieldCube, TraverseError>;
}

pub struct DefaultFieldCubeBuilder;
```

8.2 CollapsePlan

Der CollapsePlan ist keine freie To-do-Liste. Er ist eine replaybare Sequenz von Schritten mit Eingangsbezug, erwarteter Wirkung und Failure Mode.

Listing 8: CollapsePlan

```
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct CollapsePlan {
    pub id: String,
    pub field_cube_id: String,
    pub steps: Vec<CollapseStep>,
    pub ordering_policy: OrderingPolicy,
    pub expected_reduction: f64,
    pub evidence: Vec<EvidenceRef>,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct CollapseStep {
    pub id: String,
    pub kind: CollapseStepKind,
    pub target_nodes: Vec<NodeId>,
```

```

    pub required_constraints: Vec<String>,
    pub expected_effect: CollapseEffect,
    pub failure_policy: FailurePolicy,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum CollapseStepKind {
    ResolveDimension,
    ApplyConstraint,
    SplitComponent,
    MergeComponent,
    DetectPathExcision,
    MigrateCarrier,
    VerifyCandidate,
    CommitCandidate,
}

```

8.3 Ordering Policy

Das MVP verwendet eine deterministische Sortierung:

1. harte Constraints vor weichen Constraints;
2. Knoten mit hoher Kopplung vor isolierten Knoten;
3. Boundary- und PathExcision-Checks vor Solver-Ausfuehrung;
4. Verifikationsschritte vor Commit-Schritten;
5. bei Gleichstand lexikographische ID-Sortierung.

Spaeter kann diese Policy durch Fiedler-Schnitt, Betti-Luecken, Kuramoto-Kohaerenz oder TRITON-Singularitaeten ersetzt oder ergaenzt werden.

9 Optionalitaet und Pfadexzision

9.1 Formale vs. operative Option

Eine formale Option ist ein Wert, der typologisch im Wertebereich liegt. Eine operative Option ist ein Wert, der ueber mindestens einen zulaessigen Pfad erreichbar ist.

Listing 9: PathExcision

```

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct PathExcision {
    pub dimension_id: String,
    pub formal_value: String,
    pub reason: String,
    pub missing_requirements: Vec<String>,
    pub blocked_by_constraints: Vec<String>,
    pub operational_impact: OperationalImpact,
    pub evidence: Vec<EvidenceRef>,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum OperationalImpact {
    Low,

```

```

Medium,
High,
Critical,
}

```

9.2 Algorithmus

Listing 10: Path-Excision-Algorithmus

```

for dimension in sorted(field_cube.dimensions):
    for value in sorted(dimension.values):
        formal_ok = type_check(value)
        if !formal_ok: continue
        paths = admissible_paths(current_state, target=(dimension, value))
        if paths.is_empty():
            emit PathExcision(dimension, value)
        else:
            mark OperationalOption(dimension, value, best_path(paths))

```

10 Carrier und Phase-Ladder

Im MVP sind Carrier keine physikalischen Objekte, sondern operative Bearbeitungsmodi. Sie verhindern Drift, indem Schritte explizit einer Phase zugeordnet werden.

Listing 11: CarrierState

```

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct CarrierState {
    pub id: String,
    pub kind: CarrierKind,
    pub load: f64,
    pub saturation: f64,
    pub friction: f64,
    pub shock: f64,
    pub coherence: f64,
    pub status: CarrierStatus,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub enum CarrierKind {
    Analysis,
    Decomposition,
    Solve,
    Verify,
    Commit,
    Coagula,
}

```

10.1 Migration

Eine Migration ist zulaessig, wenn ein Carrier ueberlastet oder driftgefaehrdet ist und ein Zielcarrier stabiler ist.

Listing 12: CarrierMigrationReport

```
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct CarrierMigrationReport {
    pub source_carrier: String,
    pub target_carrier: String,
    pub trigger: MigrationTrigger,
    pub load_delta: f64,
    pub pre_coherence: f64,
    pub post_coherence_estimate: f64,
    pub accepted: bool,
}
```

11 Gate, Dual-Fabric und Fail-Closed

11.1 GateReport

Listing 13: GateReport

```
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct GateReport {
    pub candidate_id: String,
    pub passed: bool,
    pub checks: Vec<GateCheck>,
    pub mci: Option<f64>,
    pub pse_commit_ready: bool,
    pub failure_policy: Option<FailurePolicy>,
}

#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct GateCheck {
    pub id: String,
    pub passed: bool,
    pub severity: GateSeverity,
    pub evidence: Vec<EvidenceRef>,
    pub message: String,
}
```

11.2 Fail-Closed-Regel

Fail-Closed-Invariante

Wenn ein Candidate ein Gate nicht besteht, darf das System keinen Commit erzeugen. Es muss stattdessen eine der folgenden Ausgaben erzeugen: RefinementRequest, BoundaryReport, PathExcision, NormGap, CarrierMigrationReport oder AbortReport.

11.3 Dual-Fabric im MVP

Im MVP wird Dual-Fabric einfach implementiert:

1. Primal Check: Kandidat erfuehlt seine direkten Constraints.
2. Mirror Check: Entfernt man den Kandidaten, darf der Plan nicht besser, widerspruchsfreier oder constraint-sparsamer werden.

3. MCI: Anteil uebereinstimmender Pruefresultate zwischen Primal und Mirror.

Spaeter kann MCI ueber echte PSE-dual-Operatorik, Graphinversion oder trans-
versale Carrier berechnet werden.

12 PSE-Bruecke

12.1 Prinzip

Der Traversal Agent darf PSE nicht umgehen. Kandidaten, die als strukturelle Ereignisse gelten sollen, werden in Observations uebersetzt und ueber `macro_step` an PSE uebergeben. Nur PSE erzeugt echte `SemanticCrystals`.

Listing 14: CrystalCommitter Trait

```
pub trait CrystalCommitter {
    fn commit_candidate(
        &mut self,
        candidate: &Candidate,
        evidence_payloads: &[Vec<u8>],
    ) -> Result<CommitOutcome, TraverseError>;
}

pub enum CommitOutcome {
    Crystal(pse_types::SemanticCrystal),
    NoCrystal { gate_snapshot: Option<pse_types::GateSnapshot>, reason: String },
    EvidenceOnly(EvidenceReport),
}
```

12.2 PseMacroStepCommitter

Listing 15: PSE Committer Skeleton

```
pub struct PseMacroStepCommitter<A: pse_graph::ObservationAdapter> {
    pub state: pse_core::GlobalState,
    pub config: pse_types::Config,
    pub adapter: A,
}

impl<A: pse_graph::ObservationAdapter> CrystalCommitter for PseMacroStepCommitter<A> {
    fn commit_candidate(
        &mut self,
        candidate: &Candidate,
        evidence_payloads: &[Vec<u8>],
    ) -> Result<CommitOutcome, TraverseError> {
        let payloads = candidate.to_observation_payloads(evidence_payloads)?;
        match pse_core::macro_step(&mut self.state, &payloads, &self.config, &self.adapter) {
            Ok(Some(crystal)) => Ok(CommitOutcome::Crystal(crystal)),
            Ok(None) => Ok(CommitOutcome::NoCrystal {
                gate_snapshot: self.state.last_gate.clone(),
                reason: "pse_macro_step_returned_none".to_string(),
            }),
            Err(e) => Err(TraverseError::PseCommit(e.to_string())),
        }
    }
}
```

13 Evidence, Replay und Canonical Output

13.1 Canonical Serialization

Die neue Crate muss eine eigene canonical serialization fuer Traversal-Artefakte anbieten oder PSEs bestehende JCS-Funktion wiederverwenden. Empfehlung: wenn `pse_types::canonical_bytes` oeffentlich verfuegbar ist, verwenden; sonst in `pse-traverse` eine duenne Wrapper-Funktion definieren.

Listing 16: Canonical Helpers

```
pub fn canonical_bytes<T: serde::Serialize>(value: &T) -> Result<Vec<u8>, TraverseError> {
    > {
        serde_jcs::to_vec(value).map_err(|e| TraverseError::Canonical(e.to_string()))
    }
}

pub fn content_address<T: serde::Serialize>(value: &T) -> Result<[u8; 32], TraverseError> {
    use sha2::{Digest, Sha256};
    let bytes = canonical_bytes(value)?;
    let mut hasher = Sha256::new();
    hasher.update(bytes);
    Ok(hasher.finalize().into())
}
```

13.2 Run Descriptor

Jeder Traversal Run braucht einen eigenen Descriptor.

Listing 17: TraversalRunDescriptor

```
#[derive(Clone, Debug, Serialize, Deserialize, PartialEq)]
pub struct TraversalRunDescriptor {
    pub run_id: String,
    pub problem_spec_hash: String,
    pub operator_versions: BTreeMap<String, String>,
    pub seed: Option<u64>,
    pub ordering_policy: OrderingPolicy,
    pub pse_config_hash: Option<String>,
    pub started_at_logical: u64,
}
```

14 CLI fuer den Coding-Agenten

Das erste ausfuehrbare Tool sollte `pse-traverse-cli` sein.

Listing 18: CLI-Zielbefehle

```
cargo run --release -p pse-traverse-cli -- \
    inspect --problem examples/problem_minimal.json

cargo run --release -p pse-traverse-cli -- \
    plan --problem examples/problem_minimal.json --out target/traverse/plan.json

cargo run --release -p pse-traverse-cli -- \
    run --problem examples/problem_minimal.json --out target/traverse/run_report.json
```

```
cargo run --release -p pse-traverse-cli -- \
  replay --run target/traverse/run_report.json
```

15 Beispiel-ProblemSpec

Listing 19: examples/problem_minimal.json

```
{
  "id": "example.software.module.v1",
  "title": "Minimaler Modulplan",
  "objective": "Erzeuge einen replaybaren Plan fuer eine Rust-Crate mit klarer API.",
  "domain": "software_synthesis",
  "inputs": [
    { "id": "readme", "kind": "text", "path": "README_PSE.md" }
  ],
  "constraints": [
    {
      "id": "c.determinism",
      "kind": "hard",
      "predicate": "canonical_outputs_must_be_sorted",
      "weight": 1.0
    },
    {
      "id": "c.no_network_core",
      "kind": "hard",
      "predicate": "core_has_no_network_dependency",
      "weight": 1.0
    }
  ],
  "dimensions": [
    {
      "id": "d.crate_layout",
      "label": "Crate Layout",
      "kind": "Enum",
      "values": { "Enum": ["single_crate", "split_crates_later"] },
      "required": true,
      "source": "user"
    }
  ],
  "desired_outputs": [
    { "id": "o.plan", "kind": "collapse_plan" },
    { "id": "o.report", "kind": "fail_closed_report" }
  ],
  "risk_policy": { "fail_closed": true, "allow_oracle": false },
  "replay": { "seed": 42, "canonical": true },
  "metadata": {}
}
```

16 Phasenplan fuer die Implementierung

16.1 Phase 0: Repository-Audit

1. cargo build --workspace ausfuehren.
2. Aktuelle Crates und Workspace-Members pruefen.

3. Existenz von `pse_types::canonical_bytes`, `pse_types::content_address`, `pse_core::macro_step`, `pse_core::GlobalState` verifizieren.
4. Keine semantische Aenderung an bestehenden Tests.

16.2 Phase 1: pse-traverse MVP

1. Neue Crate `crates/pse-traverse` anlegen.
2. Workspace `Cargo.toml` ergaenzen.
3. Typen implementieren: `ProblemSpec`, `FieldCube`, `DoFGraph`, `CollapsePlan`, `PathExcision`, `GateReport`, `TraversalRunDescriptor`.
4. Deterministische Builder implementieren.
5. Unit Tests fuer canonical hashing und stabile Reihenfolge.

16.3 Phase 2: CLI

1. `tools/pse-traverse-cli` anlegen.
2. Subcommands `inspect`, `plan`, `run`, `replay` implementieren.
3. JSON-Reports unter `target/traverse/` erzeugen.
4. Beispiel-`ProblemSpec` hinzufuegen.

16.4 Phase 3: PSE-Bruecke

1. `CrystalCommitter` und `PseMacroStepCommitter` implementieren.
2. `Candidate-to-Observation-Payload-Serialisierung` definieren.
3. Wenn PSE kein Crystal erzeugt, `NoCrystal` mit `last_gate` und `Reason` ausgeben.
4. Keine synthetischen `SemanticCrystals` ohne PSE-Commit erzeugen.

16.5 Phase 4: Normen, Dual-Fabric, Collapse Certificate

1. `NormSpec`, `NormFitness`, `ConstraintLattice` einfuehren.
2. `MciGate` implementieren.
3. `CollapseCertificate` definieren.
4. Reports content-addressed speichern.

16.6 Phase 5: Agent Harness

1. `Solver Trait` stabilisieren.
2. `NullSolver`, `TemplateSolver` implementieren.
3. `OracleSolver` nur als Interface, nicht als Netzwerkimplementierung.
4. Kontextprojektionen fuer spaetere LLM-Agenten erzeugen.

17 Testplan

Test	Erwartung	Phase
canonical_same_input_same_hash	Gleiches ProblemSpec erzeugt gleichen Hash.	1
canonical_map_order_stable	Unterschiedliche JSON-Key-Reihenfolge erzeugt gleiche canonical bytes.	1
fieldcube_builds_from_minimal_spec	Minimal-Beispiel erzeugt FieldCube.	1
detects_path_excision	Formaler Wert ohne Pfad erzeugt PathExcision.	1
collapse_plan_stable_order	Gleiche Inputs erzeugen gleiche Step-Reihenfolge.	1
cli_plan_writes_report	CLI erzeugt Plan-JSON.	2
pse_bridge_returns_no_crystal	Keine Crystal-Bildung fuehrt zu NoCrystal, nicht Panic.	3
gate_fail_is_fail_closed	Gate-Fail erzeugt Refinement/Abort, keinen Commit.	3
mci_gate_bounded	MCI liegt in [0,1].	4
replay_report_matches	Run Report ist replaybar.	2/4

18 Definition of Done

18.1 MVP-konform

Eine Implementierung gilt als MVP-konform, wenn:

1. `cargo test --workspace` erfolgreich ist;
2. `cargo run --release -p pse-traverse-cli -- plan --problem examples/problem_minimal.json` einen deterministischen Plan erzeugt;
3. derselbe Plan bei erneutem Lauf byte-identisch ist;
4. PathExcision-Erkennung getestet ist;
5. Gate-Fail nie zu Commit fuehrt;
6. PSE-Bruecke compilebar ist oder hinter Feature Flag `pse-commit` sauber deaktiviert werden kann;
7. alle neuen public types dokumentiert sind;
8. keine Netzwerkabhaengigkeit im Kern existiert.

18.2 v0.1-konform

Eine Implementierung gilt als v0.1-konform, wenn zusaetzlich:

1. `PseMacroStepCommitter` mit `pse_core::macro_step` arbeitet;
2. `TraversalRunDescriptor` und `Reports` content-addressed sind;
3. `replay` Subcommand mindestens Hash- und Planstabilitaet verifiziert;
4. `GateReport` und `CollapsePlan` `EvidenceRefs` fuehren;
5. `NoCrystal` als legitimer, diagnostischer Ausgang behandelt wird.

19 Masterprompt fuer den Coding-Agenten

Listing 20: Agentenprompt

Du arbeitest in einem Rust-Monorepo, das PSE enthaelt. Ziel ist die Implementierung eines post-symbolischen Traversierungsagenten mit ueberpruefbarer Historie. Implementiere keine neue Theorieprosa, sondern kompilierbaren, getesteten Code.

Architekturregel:

- Bestehenden PSE-Kern nicht vermischen.
- Neue Crate crates/pse-traverse anlegen.
- Optional ein Tool tools/pse-traverse-cli anlegen.
- PSE bleibt Commit- und Evidence-Kern.
- Traversal erzeugt ProblemSpec, FieldCube, DoFGraph, CollapsePlan, PathExcision, GateReport und CommitOutcome.

MVP-Aufgaben:

1. Fuege crates/pse-traverse zum Workspace hinzu.
2. Implementiere Typen: ProblemSpec, FieldCube, DimensionSpec, ConstraintSpec, DoFGraph, CollapsePlan, PathExcision, Candidate, GateReport, TraversalRunDescriptor.
3. Nutze serde, serde_json, serde_jcs, sha2 und BTreeMap/BTreeSet fuer deterministische canonical outputs.
4. Implementiere DefaultFieldCubeBuilder.
5. Implementiere detect_path_excision.
6. Implementiere DefaultCollapsePlanner mit stabiler Sortierung.
7. Implementiere GateEngine mit Fail-Closed-Verhalten.
8. Implementiere CrystalCommitter Trait und PseMacroStepCommitter.
9. Lege examples/problem_minimal.json an.
10. Lege tools/pse-traverse-cli mit inspect/plan/run/replay an.
11. Schreibe Unit Tests fuer Determinismus, PathExcision, Planstabilitaet, Fail-Closed und PSE-Bridge-NoCrystal.

Strikte Verbote:

- Keine nichtdeterministischen HashMaps in canonical outputs.
- Keine Netzwerkabhaengigkeiten im Kern.
- Keine SemanticCrystals frei erzeugen, wenn PSE sie nicht committed.
- Keine Panics auf normalen Gate-Fails.
- Keine Architekturdrift in pse-core, ausser absolut notwendige re-exports.

Akzeptanz:

- cargo fmt
- cargo test --workspace
- cargo run --release -p pse-traverse-cli -- plan --problem examples/problem_minimal.json
- Zweiter identischer Lauf erzeugt byte-identischen Report.

A Maschinenlesbarer Minimalvertrag

Listing 21: PSE Traversal Agent v0.1 Contract

```
PSE_TRAVERSAL_AGENT_v0_1 := {
  Input: ProblemSpec | ObservationStream | Corpus | ArtifactSet,
  CoreState: FieldCube(D,A,K,G,P,Lambda,kappa,H),
  WorkGraph: DoFGraph,
```

```

RequiredOperators: [Project, BuildDoFGraph, PlanCollapse,
                    DetectPathExcision, Solve, Gate, Coagula, Commit],
PSEBinding: macro_step | EvidenceOnly,
Outputs: CollapsePlan | GateReport | PathExcision | CommitOutcome |
        TraversalRunReport,
Guarantees: [DeterministicSerialization, ReplayableReports,
             FailClosedGate, EvidenceRefs, NoUngatedOracleEmission],
NonGoals: [CovertManipulation, UnboundedContextClaim,
           FreeSyntheticSemanticCrystals]
}

```

B Glossar

Begriff	Bedeutung
ProblemSpec	Normierter Eingang fuer den Agenten. Kein Prompt-Ersatz, sondern strukturierte Problemdefinition.
FieldCube	Erweiterter Hypercube mit Dimensionen, Constraints, Kopplungen, Pfaden, Carriern und Evidence.
DoFGraph	Freiheitsgradgraph des Problems.
PathExcision	Formale Option ohne admittierbaren operativen Pfad.
Carrier	Operativer Bearbeitungsmodus oder Träger von Last, Kontext oder Commit-Funktion.
CollapsePlan	Deterministische Sequenz zur Reduktion des Freiheitsgradraums.
Candidate	Vom Solver erzeugter moeglicher Teilausgang.
GateReport	Pruefobjekt, das Pass/Fail und Evidence pro Check festhaelt.
Coagula	Fail-Verarbeitung, die aus einem Scheitern eine strukturelle Folgeaktion erzeugt.
SemanticCrystal	PSE-eigener content-addressed Strukturcommit.
CommitOutcome	Ergebnis eines Commitversuchs: Crystal, NoCrystal oder EvidenceOnly.
TraversalRunDescriptor	Replay-faehiger Descriptor eines Traversierungslaufs.

C Schlussbemerkung

Diese Spezifikation verengt die vorherige Theorie bewusst auf den naechsten implementierbaren Schritt. Der Agent muss zuerst beweisen, dass er Problemraeume deterministisch modellieren, Pfadexzisionen erkennen, CollapsePlaene stabil erzeugen und PSE als Commit-Kern anbinden kann. Erst danach lohnt sich die Erweiterung zu Norm-Lifecycle, echten LLM-Solvern, Multi-Agent-Swarm oder GUI. Der richtige Weg ist daher: kleiner Kern, harte Tests, replaybare Reports, dann rekursive Erweiterung.